Multi-Task Learning for Stock Price Prediction

Computational Intelligence (CIS*4780) - March 2026

Stella Rovazzi
srovazzi@uoguelph.ca

Matthew Lawford
mlawford@uoguelph.ca

Julia Horner
jhorne03@uoguelph.ca

Abstract

As a core branch of financial forecasting, predicting stock prices plays a crucial role for many business leaders, investors, and policymakers in optimizing investment strategies. Financial markets generate vast amounts of highly volatile, sequential data, making them a challenging target for traditional forecasting methods. While traditional, single-task models offer potential solutions, many models struggle to balance historical pattern recognition and intra-sector relationships. To address this, an LSTM and an Extra Trees using Multi-Tasking models were implemented using PyTorch and R to process sequential market data and predict future movements. By fine-tuning parameters based on the respective models, they were able to capture the data's underlying patterns, achieving precise convergence. Ultimately, this approach highlights the potential efficacy of multi-tasking learning in uncovering hidden market dynamics, creating new paths for new investment models.

I. Introduction

Forecasting of financial time series is a significant challenge in computational intelligence due to the high volatility, non-linearity and inconsistent variety of global markets [**?**], [1]. Traditional univariate models often try to predict stock price movements in isolation, not accounting for the deep rooted dependencies and shared behaviours between firms in the same sector. This project investigates Multi-Task Learning, a form of inductive transfer to improve predictive accuracy by explicity modeling these interconnections [2].

By utilizing shared representations, the Multi-Task Learning framework forces the model to favor parameters that generalize across different but associated market behaviours [3]. This approach leverages sector-wide knowledge to improve the performance of individual target tasks. Our study focuses on a diverse cohort of 25 companies spanning the hardware and software sectors, derived from the iShares S&P/TSX Capped Information Technology Index and Vanguard Information Technology ETFs.

The primary objective is to develop a robust architecture capable of capturing complex, non-linear dependencies in financial data from 2009 to 2017. To evaluate the efficacy of the Multi-Task Learning approach, we compare two sophisticated architectures:

- Long Short-Term Memory (LSTM) Networks: A recurrent architecture designed for sequential data, implemented with a shared hidden layers and task-specific output heads.
- Multi-Task Extra Trees: An ensemble method utilizing multi-output regression to optimize node splits across all tasks simultaneously.

By comparing these multi-tasking frameworks against traditional single-prediction baselines, this study demonstrates how shared representations can reduce variance and provide more stable price forecasts in volatile financial environments.

II. Motivation and Objectives

Motivation

Stock market data is volatile and inherently non-linear, meaning its values change quickly and drastically, with inconsistent variance. This makes accurately forecasting stock prices a challenge since many methods are too simple to capture this behaviour. Neural nets and decision trees are currently used to capture stock market behaviour, but, according to the research, they predominantly focus on predicting a single stock at a time [4]. Stocks within the same sector likely affect or reveal patterns in each other's data. This paper explores whether this correlation can be used to improve the predictive performance of current non-linear models for stock price prediction.

Objectives

The primary objective of this research is to evaluate whether a multi-task learning framework can improve the predictive accuracy of individual stock prices by leveraging historical information from correlated stocks within the same sector. To achieve this, the project also contains the following sub-objectives:

- Comparative Performance Analysis: To perform a good comparison between multi-task learning architectures and traditional single-task (univariate) baselines to determine if modeling stocks in isolation fails to capture critical sector-wide dependencies.
- Validation of Inductive Transfer: To assess the effectiveness of inductive transfer, specifically the ability to improve the performance of a target task after learning to model related tasks within the technology sector.
- Evaluation of Inductive Bias: To determine if forcing the model to favour parameters that are effective across multiple target tasks leads to more stable forecasts in highly volatile financial environments.
- Cross-Model Generalization: To apply the multi-task learning approach across 2 distinct computational intelligence architectures, Long Short-Term Memory (LSTM) networks and Multi-Task Extra Trees to ensure the findings are robust across different algorithmic types.
- Sector Interrelationship Testing: To validate if the shared representation between hardware and software firms provides a accuracy boost by modeling their interconnections rather than modeling them in isolation.

III. Methodology

Data Availability: Given the abundance of detailed historical stock market data, the dataset was carefully chosen. A mix of hardware and software companies between 2009 and 2017 was selected to ensure that the data exhibits behaviours uninfluenced by external severe market-changing events. Training on this data allowed for the architecture to capture the complex, non-linear market behaviors that traditional models struggle to represent. The timeframe was specifically restricted to the period from 2009 through 2017 to avoid two major economic anomalies: the 2008 financial crash and the COVID-19 pandemic. Excluding these catastrophic events prevents the model from unintentionally being trained on massive market disruptions.

Datasets: The specific companies in the dataset were taken from the iShares S&P/TSX Capped Information Technology Index ETF and the Vanguard Information Technology ETF. These professionally curated funds were used to ensure a baseline of market viability and relevance. To guarantee sufficient training data, a total of 25 companies were selected: 5 from iShare and 20 from the Vanguard ETF. The selected companies were subsequently divided into three categories based on their market capitalization: large-cap (greater than $120,000,000 USD), mid-cap (between $120,000,000 and $50,000,000 USD), and small-cap (less than $50,000,000 USD).

Intra-Sector Relationships and Multitasking: A varied composition of technology companies, spanning across a large range of valuations, was deliberately selected to optimize the architecture for multi-task learning. This introduced the models to complex intra-sector relationships, allowing them to learn how behaviours in one sector influence or correlate with another. By multi-tasking across these datasets, the model avoids overfitting to a single company size and develops a stronger understanding of the underlying patterns of the market.

A. Algorithms

Long Short-Term Memory(LSTMs): The Long Short-Term Memory (LSTM) architecture was chosen for its specialized ability to model long-range temporal dependencies in financial time series, which effectively overcomes the vanishing gradient problem common in standard Recurrent Neural Networks (RNNs) [5]. In our Multi-Task framework, each input sequence $X \in \mathbb{R}^{L \times D}$ represents $L = 60$ temporal lags across $D = 25$ features (ticker log-prices). These inputs are processed through gated units—comprising input, forget, and output gates, to regulate the flow of information across time steps.

To evaluate the efficiency of shared representations, we implemented a hard parameter sharing multi-task framework [2]. As illustrated in Figure 1:

Utilizing the pytorch framwork, the architecture was constructed with the following components:

- **Shared Hidden Layers:** Two stacked LSTM layers with 128 hidden units each. These layers serve as a common feature extractor, learning a global representation of the technology sector's dynamics by pooling data from all 25 companies.

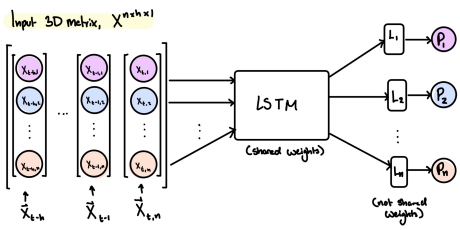

Fig. 1. Diagram of hard parameter sharing and inputs for an LSTM Multi-Tasking model. (Drawn by group).

- **Task-Specific Heads:** 25 independent linear output layers (heads) that branch from the final LSTM hidden state. Each head is responsible for the regression of a specific stock's log-price, allowing the model to specialize its final prediction for each ticker while utilizing shared temporal features.

During training, the model optimizes a composite loss function to balance performance across all tickers:

$$\mathcal{L}_{total} = \sum_{t=1}^{25} \text{MAE}_t \tag{1}$$

where MAE_t represents the Mean Absolute Error for a specific company's prediction. This joint optimization acts as a powerful regularizer, because the shared weights must satisfy the trends of the entire cohort, the model is less likely to overfit to idiosyncratic noise or high-frequency volatility in a single stock's history [6].

Multi-Tasking Extra Trees (or Extremely Randomized Trees): Extra Trees (ET) is a tree ensemble method like Random Forests, but it produces more decorrelated trees. It uses the entire data set to build decision trees, rather than bootstrapping as in random forests. It also differs in its threshold selection, which randomly selects where to split the feature rather than using an optimization method. It still chooses m_{try} number of random features to choose the feature splits to compare, where $m_{try} = \lfloor \frac{p}{3} \rfloor$ or $m_{try} = \lfloor \sqrt{p} \rfloor$ and p is the number of features. There are two relevant extensions to this algorithm that make it similar to our LSTM hard parameter multi-tasking [7].

The first is introducing multi-output, which essentially means the loss function is the summation of sample error and task outputs. This effectively makes each node split consider the error of the candiate splits' effect across all tasks [7]. The second is introducing task-focused splits, or the ability to make decisions based on what the task is being predicted. To do this, a random group of m_{try} features and S random possible subsets of tasks are selected (S$\epsilon\mathbb{Z} < T$ randomly selected). Each feature split is randomized, and the multi-output error is calculated. Then, f_k (Equation 2) is calculated for each candidate task subset or feature; this is the prediction. The error is calculated from the prediction and used to choose the best split at the node [7].

$$f_k = \frac{\sum_{\nu \epsilon I_k} y_\nu + \alpha \frac{1}{\lfloor I \rfloor} \sum_{w \epsilon I} y_w}{\lfloor I_k \rfloor + \alpha} \tag{2}$$

"I denotes the set of sample indexes at the current node, I_k the set of sample indexes of the task k at the current node and α is a regularization parameter weighting the influence of the tasks with default value 1. Then, the task cut point is randomly selected from the interval $(\min_k f_k, \max_k f_k)$" - Schmid et. al [7]

To implement this method, we used R, since there is a pre-made R package for multi-tasking tree enssemblers. It is technically archived in newer versions of R, but can be downloaded here. This was decided because the paper explaining the metholody used this package [7].

B. Metrics and Evaluation

Rolling Cross-Validation: To validate the model model, rolling cross validation was used. This method slides a set length "window" along the folds, validating with the last fold of the window. A visual for this method is shown in figure 2. It's ideal for time series data [8].

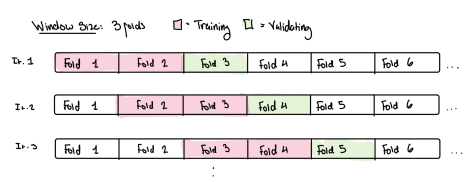

Fig. 2. Rolling cross validation explination. (Drawn by Group)

Mean Absolute Error (Criterion and Metric): Mean Absolute Error (MAE) is a metric robust to outliers [8]. For that reason, the MAE was chosen as the loss function, and the models were evaluated with this metric. The MAE could not be used as the "criterion" or error to minimize in extra trees because the R package did not support it.

Training and Evaluation: The data was split into 80-20 training and evaluation. This made the evaluation period approximately cover the years 2015 to 2017.

C. Preprocessing

Misaligned Dates: Due to different market holidays, some days in the data included both null values and recorded prices. The days with null values were removed to keep the dataset consistent. Dropping a few days of data should not obscure the stocks' growth patterns and relationships.

Logarithmic Scale: The data was scaled logarithmically. This was to ensure stable predictions by reducing effect of outliers on the model and to account for the exponential patterns in each stock's data. The predictions become more stable because most analysis techniques assume additive data, including machine learning. [9].

The scaling affect is shown in appendix A, the medium stock ACIW is on a similar scale to the large stock AAPL after logging the data.

Lag Length: Lags are data from a previous date; for example, lag 1 at time t is the data point at time $t-1$ [8]. This idea is from time series, where it is assumed that lags of the data have some affect on the current t's data.

For this model, 60 lags were used because a quarter of the fiscal year is approximately this many days [10]. This value considers only days when the markets are actually open.

LSTMs: Pytorch requires LSTM inputs to be 3D tensors [11]. The data looks at stocks, with the only attribute being the stock price. Referring to figure 1, the matrix on the left would be an example 3D tensor input, with each vector $\vec{x}$ representing all the stocks of interest at the given t. There is no third dimension depicted, as the only attribute of each stock of interest is the given price at time t.

The times within the input are $\vec{x_{t,n}}$ and the lags of $\vec{x_{tn}}$, which represent our rolling window, since $\vec{x_{t+1}}$ should use all the previous lags for its prediction.

Extra Trees: To turn the stock price data into an acceptable input for a tree ensemble, the lags must be added manually. Each datapoint will then have 60 attributes representing the 60 associated lags lags [12]. Days without all 60 previous lags have been dropped.

D. Building the Models

LSTMs: This algorithm was implemented in PyTorch, which provides native LSTM modules and facilitates multi-task learning. The models in this paper were tuned based on the following hyperparameters: epochs, learning rate, hidden nodes, hidden layers, and batch size. Fine-tuning these five parameters allowed the models to capture the data's underlying patterns, ensuring efficient, precise convergence while actively preventing overfitting.

The development of the LSTM architecture followed a progressive tuning strategy, moving from a univariate baseline to a complex hard parameter-sharing multi-task framework. Both models were optimized using a heuristic-based grid search (a targeted manual search) to identify the best spot between model capacity and generalization. The primary objective was to move beyond naive forecasting, where the model merely mirrors the price at $t-1$ and capture the underlying sector-wide volatility.

The models were systematically tuned across five key dimensions to ensure precise convergence and prevent overfitting: Network depth and capacity: Tested between 1-6 layers and 25-200 hidden nodes. A 2 layer architecture with 128 nodes was found to best represent non-linear patterns without having gradient stagnation.

Optimization Parameters: The learning rate was tested from 0.01 down to 0.0001. A rate of 0.0005 with the Adam optimizer provided the most stable validation MAE across the 25-stocks.

Training Dynamics: Standardized to 50-100 epochs with a batch size of 64 to manage computational overhead while ensuring sufficient exposure to the 2,712-day training set.

The fundamental shift in the Multi-Task approach was the implementation of a Composite Loss Function. Unlike the single-task baseline, the Multi-Task LSTM aggregated the Mean Absolute Error (MAE) from all 25 task-specific heads into a single joint loss. This forced the shared LSTM layer to ignore idiosyncratic noise in individual stocks and prioritize parameters that generalized across the entire tech sector.

While dropout could have been tuned, it was maintained as a fixed regularization constraint of 0.2 rather than a variable parameter. Given the high dimensionality of 25 simultaneous tasks, a fixed dropout served as a non-negotiable guard. This ensured that the shared hidden layers remained robust against the influence of high-market-cap outliers allowing the model to learn a global representation that remained applicable to mid and small cap tickers within the index.

Extra Trees: For Extra Trees, single-task and multi-task, the parameters that could be tuned are [7]:

- The number of grown trees
- The cardinality of the sample of features at every node is m_{try}
- The number of random split values at each node
- The minimum number of observations that each terminal node should contain (stopping criteria)

The models in this paper are tuned for m_{try}, the number of random splits/cuts, and the number of terminal node observations. The number of trees was set to the default of 500 [13]. Any excess grown tree doesn't impact the predictive power of the model, and 500 is likely enough for the tasks of interest [14].

IV. Results and Findings

A. Single-Task LSTM

For the single-task LSTM, IBM and BlackBerry stocks were evaluated. The plots to tune the hyperparameters are in CHANGE TO THE CORRECT APPENDIX. The model performance was optimized based on the lowest validation Mean Absolute Error (MAE).

Hyperparameters

BlackBerry (BB.TO): For the BlackBerry dataset, the optimal validation MAE was achieved using 42 epochs, 25 hidden nodes, a learning rate of 0.0008, and two LSTM layers.

IBM: For the IBM dataset, the optimal validation MAE was achieved using 6 epochs, 100 hidden nodes, a learning rate of 0.0009, and three LSTM layers.

Algorithm Performances

BlackBerry's test evaluation of MAE was 0.0319 for single-task LSTM, and IBM's test evaluation of MAE was 0.0933.

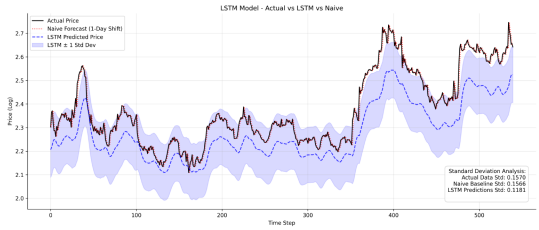

Fig. 3. Prediction of the the blackberry single-task LSTM with spread of prediciton, which is zero and has no visual.

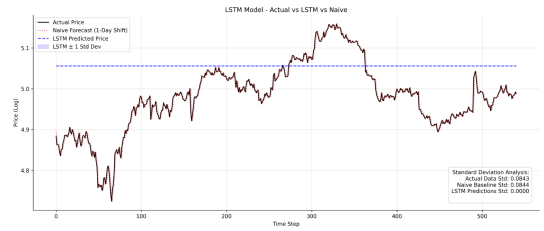

Fig. 4. Prediction of the the IBM single-task LSTM with spread of prediciton.

B. Multi-Task LSTM

The Multi-Task LSTM architecture was designed to leverage shared temporal features across a synchronized dataset of 25 stocks. By utilizing shared hidden layers, the model implements an inductive bias that favours parameters generalizing across the technology sector.

Hyperparameters: The optimal configurations for the MTL model was determined through a systematic 3 stage ablation study evaluating layer count, hidden dimension size, and learning rates (See Appendix B). The final architecture selected consisted of:

- **LSTM Layers:** 2. As illustrated in Figure B1, 2 layers provided the optimal balance between model capacity and generalization, whereas deeper architectures showed a sharp increase in validation error.
- **Hidden Dimension:** 128. The selection of 128 hidden units was justified by the performance trends in Figure B2, proving sufficient to capture high-dimensional cross-stock correlations without over-parameterization.
- **Learning Rate:** 0.0005. Figure B3 highlights that this rate allowed for the most stable descent into the convergence learning cliff, avoiding the instability found in more aggressive rates.
- **Dropout:** 0.2. Applied to the shared hidden layers to provide robust regularization against the high volatility of the technology sector.
- **Batch Size:** 64. Utilized to provide a stable gradient estimate for the weighted sum of task-specific loss functions while maintaining enough stochastic noise for effective generalization.

Algorithm Performances: The multi-task LSTM achieved a final aggregate test Mean Absolute Error (MAE) of **0.4444** across all 25 task-specific output heads. Figure D4 shows the training vs. validation convergence.

The relevant results for comparison with the single-task baselines are below:

- **BB.TO:** MAE of 0.1365.
- **IBM:** MAE of 0.1705.

Figures 5 and 6 visualize the predictions for these stocks over the 2017 evaluation period.

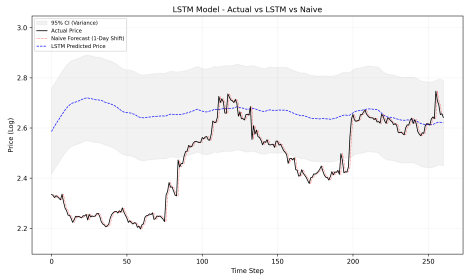

Fig. 5. Evaluation prediction of BB.TO using Multi-Task LSTM. Solid line is true value; blue dashed is prediction; and shaded area is the 1σ error variance.

The BB.TO prediction follows the actual price trend closely, particularly in the latter half of the year. The actual price remains largely within the standard deviation ribbon, indicating successful sector-trend capture by the shared LSTM layer.

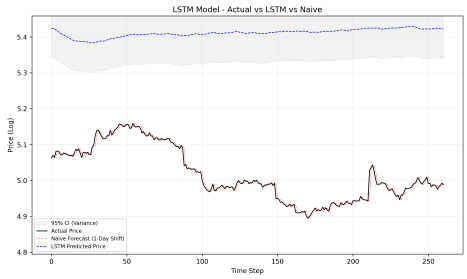

Fig. 6. Evaluation prediction of IBM using Multi-Task LSTM. Solid line is true value; blue dashed is prediction; and shaded area is the 1σ error variance.

For IBM, the Multi-Task model maintains a significant global bias. While the actual price crashed during 2017, the model remained anchored to the positive growth trend of the other 24 stocks in the cohort. This results in the actual price diving well below the 1σ error variance ribbon, highlighting the model's prioritization of sector-wide signals over idiosyncratic stock volatility.

Furthermore, a localized inverse relationship is observed in the initial 50-day window (Time Steps 0-50). While the true values exhibit a concave U-shaped recovery, the prediction demonstrates a convex n-shaped curve. This discrepancy is identified as a phase lag caused by the 60-day input window, the model's shared representations remained influenced by the positive momentum of the preceding fiscal quarter, resulting in a delayed response to the stock's independent downward trend.

The evaluations of all stocks are in Appendix E.

C. Single-Task Extra Trees

Hyperparameters

The plots showing the tuning choices are in C.

IBM: IBM Had the smallest validation MAE with terminal node size 3, 2 random cuts, and m_try size $p/3$.

Blackberry (BB.TO): Blackberry had the smallest validation MAE with terminal node size 3, 8 random cuts, and m_try size $\sqrt{p}$. IBM's evaluation MAE was 0.01502721 for single task ET.

Figure 7 shows the prediction of this model over the monthly evaluation period.

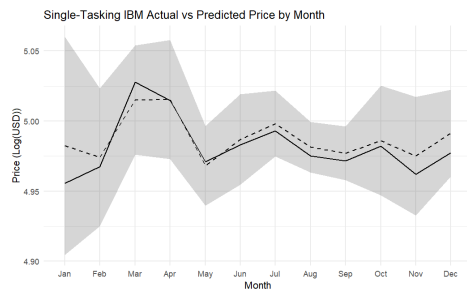

Fig. 7. Evaluation prediction of IBM stock using single-task extra tree. Solid line is true value; dotted is prediction; and shaded area is the variance.

Nearing the later part of the calendar year, the single-task model seems to predict a value higher than the IBM stock. The variance at the begining of the evaluation period is high, implying the model could be bias towards the training data.

Algorithm Evaluation Performances

Blackberry's evaluation MAE was 0.02308594 for single-task ET. Figure 8 shows the prediction of this model over the monthly evaluation period.

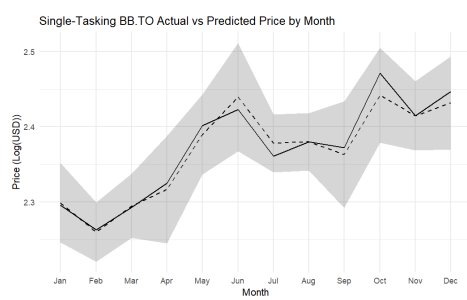

Fig. 8. Evaluation prediction of BB.TO stock using single-task extra tree. Solid line is true value; dotted is prediction; and shaded area is the variance.

The variance of the prediction is consistent throughout the evaluation period. There is a mix between over and under reporting the price of the blackbery stock.

D. Multi-Task Extra Trees

Hyperparameters

Appendix D shows the tuning plots for this section. Tuning showed 3 nodes and 5 nodes had almost equivalent MAE, so we chose 3 nodes. Tuning also showed $p/3$ was the best for m_try and 10 random cuts would be ideal for multi-tasking.

Algorithm Evaluation Performances

The relevant results for comparision are below:

- **BB.TO:** MAE of 0.015650922
- **IBM:** MAE of 0.011959280

The results for all prediction can be found in Appendix F.

Figures 10 and 9, visualizes the multi-tasked predictions of the stocks previously predicted with single-task models.

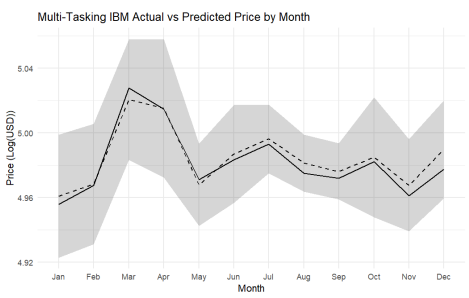

Fig. 9. Evaluation prediction of IBM stock using multi-task extra tree. Solid line is true value; dotted is prediction; and shaded area is the variance.

The IBM multi-task prediction has consistent variance throughout the evaluation period. It still, however, predicts a higher price than the true price nearing the end of the calendar year.

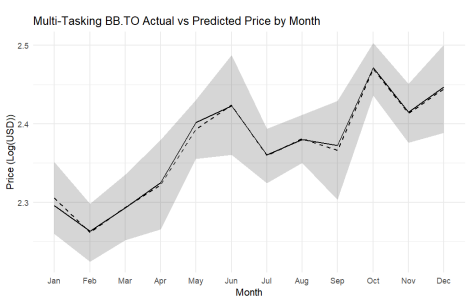

Fig. 10. Evaluation prediction of BB.TO stock using multi-task extra tree. Solid line is true value; dotted is prediction; and shaded area is the variance.

The blackberry stock prediction holds very close to the true prediction.

Multi-tasking improved the blackberry stock price prediction, while the IBM prediction essentially remained unchanged; a 0.0006 difference is negligable.

V. Discussion

A. Interpretation of Results

Single vs Multi: **INSERT PARAGRAPH FOR LSTM SINGLE VS MULTI****
The results, comparing single-task learning ET to multi-task learning ET, reveal that leveraging stock correlations within the same sector can improve predictive performance. Blackberry MAE decreased after applying multi-task learning, indicating an improvement. Though the MAE of the IBM stock didn't improve with multitasking, the prediction variance became more consistent. You can see in figure 7 how large the variance is in the early months of the calendar year; this implies the single-task was biased towards training data. Figure 9 shows that multi-tasking significantly reduces the variance, making predictions more consistent and the model more generalizable. The change in IBM's variance is due to a successful application of inductive bias and transfer, since the prediction could use the other stock's patterns to stabilize itself.

LSTM MTL vs Extra Trees MTL: Due to the naive forecasting problem, it is not possible to determine whether multi-task learning is a robust methodology across different models. This does not rule out the possibility that multi-task learning is beneficial and robust across multiple models, as the problem stems from the relationship between stock data and LSTMs rather than the multi-tasking [15].

The multitasking extra trees model properly captures sector-wide dependencies in the data and improves prediction performance. The LSTM does not. It is likely that converting the model to a multi-task learning format caused the naive forecasting problem to spread to previously unaffected predictions.

B. Advantages and Disadvantages of Multi-Tasking

Advantages:

- Faster to train than single-task learning, as you only have to tune one set of hyperparameters
-

Disadvantages:

- This is only helpful if all the stock prices are of interest. If only one stock price is of interest, multi-input to single output may be a better method.
- With LSTMs, if one or more predictions run into the naive forecasting problem it will cause all of your predictions to be worse.

VI. Conclusion

We have concluded

Future Works

** Maybe explore if dropping the bad LSTM ones make better ?? **

To further this project, the remaining stocks of interest could have their single-task counterparts calculated and compared. This would provide a better understanding of how multitasking affects the predictions.

Tthe intermediate step of multi-input single-output predictions could also be explored. This would reveal the effect of shared weights in multi-tasking, because the multi-input single-output model would use the same input data as our multi-tasking model, but it would optimize for only one prediction.

References

[1] A. Abhyankar, L. S. Copeland, and I. Wong, "Nonlinear dynamics in real-time equity indices: evidence from the uk, us, germany and japan," *The Economic Journal*, pp. 864–880, 1995.

[2] R. Caruana, "Multitask learning," *Machine learning*, vol. 28, pp. 41–75, 1997.

[3] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Transactions on knowledge and data engineering*, vol. 22, no. 10, pp. 1345–1359, 2010.

[4] W. Bao, Y. Cao, Y. Yang, H. Che, J. Huang, and S. Wen, "Data-driven stock forecasting models based on neural networks: A review," *Information Fusion*, vol. 113, p. 102616, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1566253524003944

[5] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, pp. 1735–1780, 1997.

[6] S. Ruder, "An overview of multi-task learning in deep neural networks," *Insight centre for data analytics*, 2017.

[7] L. Schmid, A. Gerharz, A. Groll, and M. Pauly, "Tree-based ensembles for multi-output regression: Comparing multivariate approaches with separate univariate ones," *Computational Statistics & Data Analysis*, vol. 179, p. 107628, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167947322002080

[8] J. Slater, "Time series analysis (stat*4360)," University of Guelph, 2025.

[9] "Decomposition of time series," 2025. [Online]. Available: https://medium.com/@m.capurso/decomposition-of-time-series-071ce7b7a5a0

[10] R. Malhotra, "How many trading days in a year?" 2026. [Online]. Available: https://www.stockgro.club/blogs/trading/how-many-trading-days-in-a-year/#:~:text=Trading%20days%20don't%20spread,a%20shorter%20month%20for%20trading.

[11] "Sequence models and long short-term memory networks," 2017. [Online]. Available: https://docs.pytorch.org/tutorials/beginner/nlp/sequence_models_tutorial.html

[12] I. P. Carrascosa, "Forecasting the future with tree-based models for time series," 2025. [Online]. Available: https://machinelearningmastery.com/forecasting-the-future-with-tree-based-models-for-time-series/

[13] "extratrees: Function for training extratree classifier or regression." [Online]. Available: https://www.rdocumentation.org/packages/extraTrees/versions/1.0.5/topics/extraTrees

[14] "The effects of the depth and number of trees in a random forest," 2025. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/the-effects-of-the-depth-and-number-of-trees-in-a-random-forest/

[15] C. M. Ellis, "Lstm time series + stock price prediction = fail," 2022. [Online]. Available: https://www.kaggle.com/code/carlmcbrideellis/lstm-time-series-stock-price-prediction-fail/notebook#Model-performance

APPENDIX

A. Raw Data

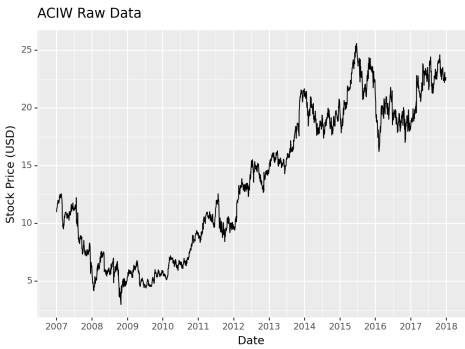

Fig. A1. Raw price data of ACIW Stock

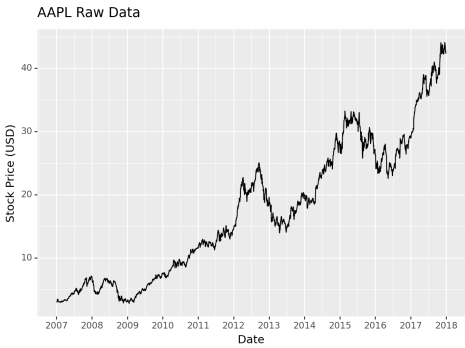

Fig. A2. Raw price data of APPL Stock

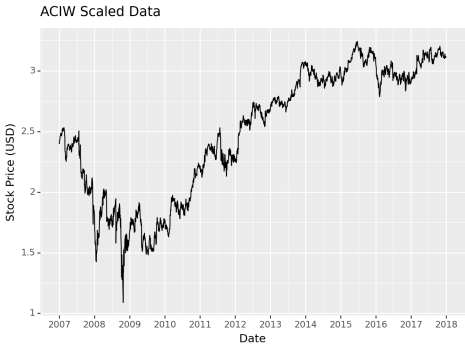

Fig. A3. Logged price data of ACIW Stock

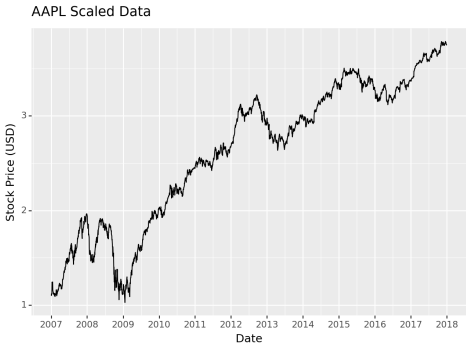

Fig. A4. Logged price data of APPL Stock

B. Multi-Task LSTM Hyperparameter Tuning

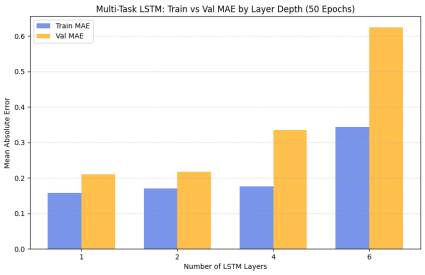

Fig. B1. Tuning Multi-Task LSTM: Number of Layers. Evaluation of 1 to 6 layers showed 2 layers provided the optimal balance of capacity and generalization.

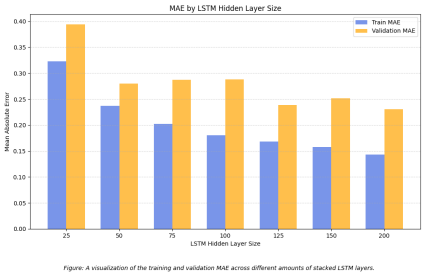

Fig. B2. Tuning Multi-Task LSTM: Hidden Dimension. Performance across a range of 25 to 200 units justified the selection of 128 hidden units.

Fig. B3. Tuning Multi-Task LSTM: Learning Rate. Comparison of rates between 0.01 and 0.0001 identified 0.0005 as the most stable for sectoral synchronization.

C. Single-Task Extra Trees Tuning

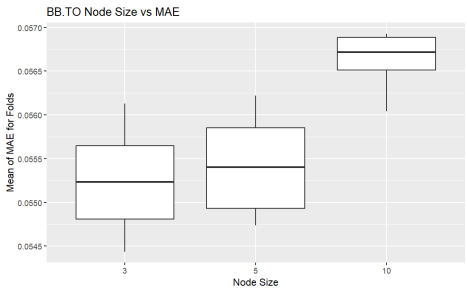

Fig. C1. Tuning Blackberry node size

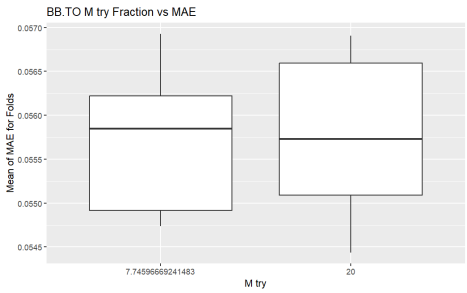

Fig. C2. Tuning Blackberry m_{try} fraction, choice between $p/3$ and $\sqrt{p}$ where p is feature length

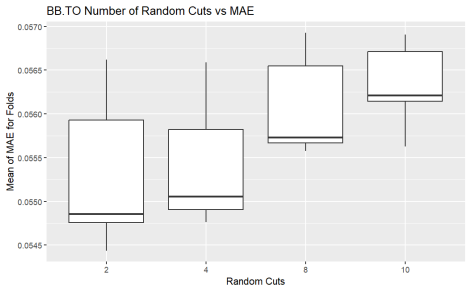

Fig. C3. Tuning Blackberry number of random cuts

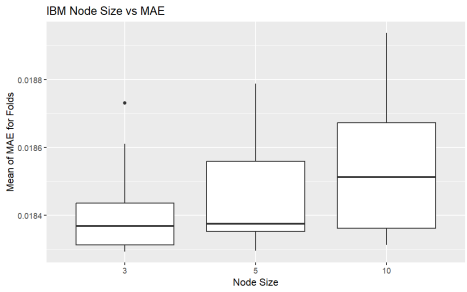

Fig. C4. Tuning IBM node size

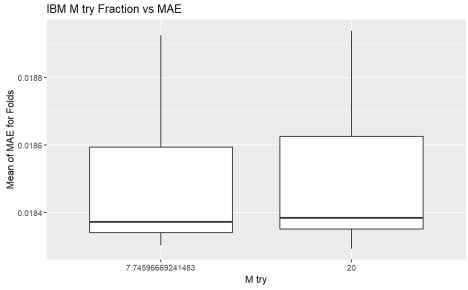

Fig. C5. Tuning IBM m_{try} fraction, choice between $p/3$ and $\sqrt{p}$ where p is feature length

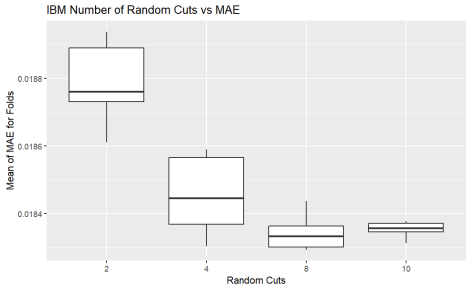

Fig. C6. Tuning IBM number of random cuts

D. Multi-Task Extra Trees Hyperparameter Tuning

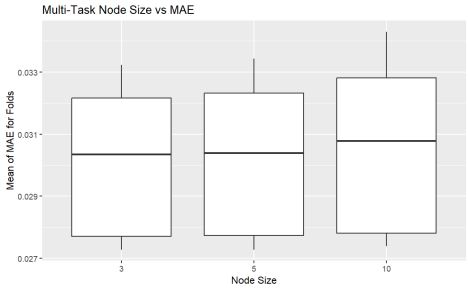

Fig. D1. Tuning Multi-Task Extra Trees node size

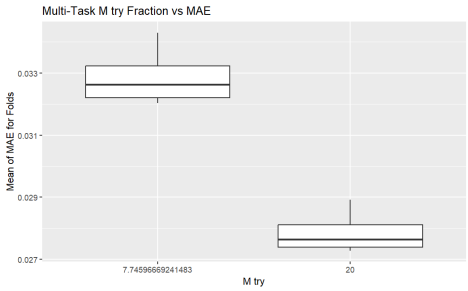

Fig. D2. Tuning Multi-Task Extra Trees m_{try} fraction, choice between $p/3$ and $\sqrt{p}$ where p is feature length

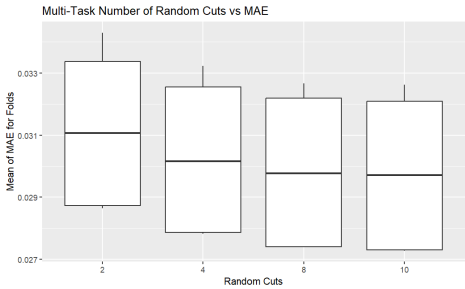

Fig. D3. Tuning Multi-Task Extra Trees number of random cuts

E. Multi-Task LSTM Results

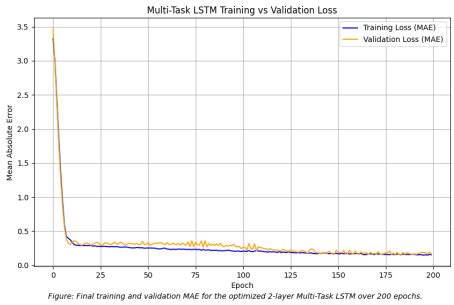

Fig. D4. Training vs. Validation MAE for the optimized 2-layer Multi-Task LSTM over 200 epochs.

Task	Mean of MAE for Evaluation Window
CSU.TO	0.1233
CLS.TO	0.1031
SHOP	0.8730
GIB-A.TO	0.0313
OTEX.TO	0.0762
DSG.TO	0.1538
KXS.TO	0.1103
BB.TO	0.1365
AMD	0.3884
IBM	0.1705
CSCO	0.0641
AAPL	0.3240
MSFT	0.2134
ORCL	0.1615
INTC	0.1026
CRM	0.2022
NOVT	0.8327
VSAT	0.1816
BDC	0.0582
SLAB	0.2605
OLED	0.7746
ACIW	0.1477
BMI	0.1815
BELFB	0.0710
BLKB	0.2459
NTCT	0.0532
UCTT	1.1013
PLAB	0.1730

F. Multi-Task Extra Trees Results

Task	Mean of MAE for Evaluation Window
AAPL	0.010017864
ACIW	0.013566978
AMD	0.031279329
BB.TO	0.015650922
BDC	0.017563812
BELF	0.019272468
BLKB	0.013493255
BMI	0.011254196
CLS.TO	0.012401716
CRM	0.013950118
CSCO	0.009417804
DSG.TO	0.011196125
GIB-A.TO	0.009294681
IBM	0.011959280
INTC	0.009181391
KXS.TO	0.017382097
MSFT	0.009562804
NOVT	0.013969065
NTCT	0.015411194
OLED	0.024107319
ORCL	0.008827831
OTEX.TO	0.009609518
PLAB	0.015337510
SLAB	0.014071373
UCTT	0.023878160
VSAT	0.012968538